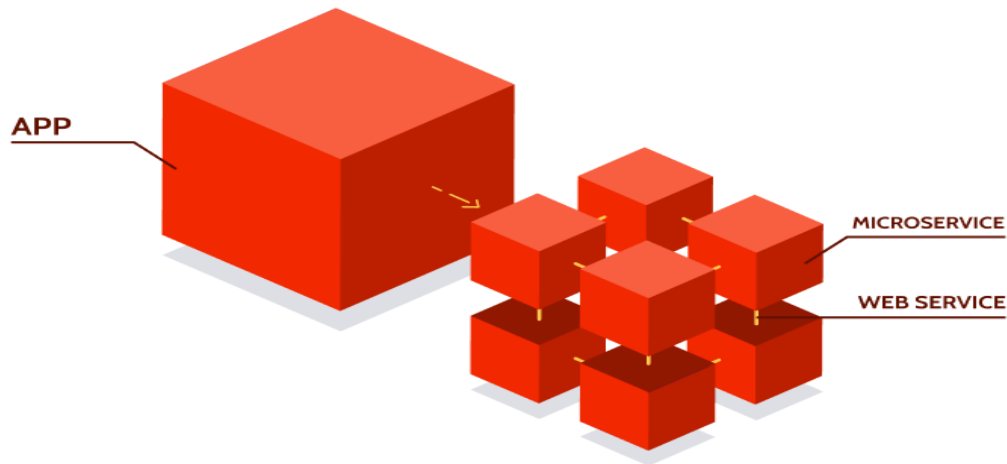




Les Micro services

La Dockerisation des micro services

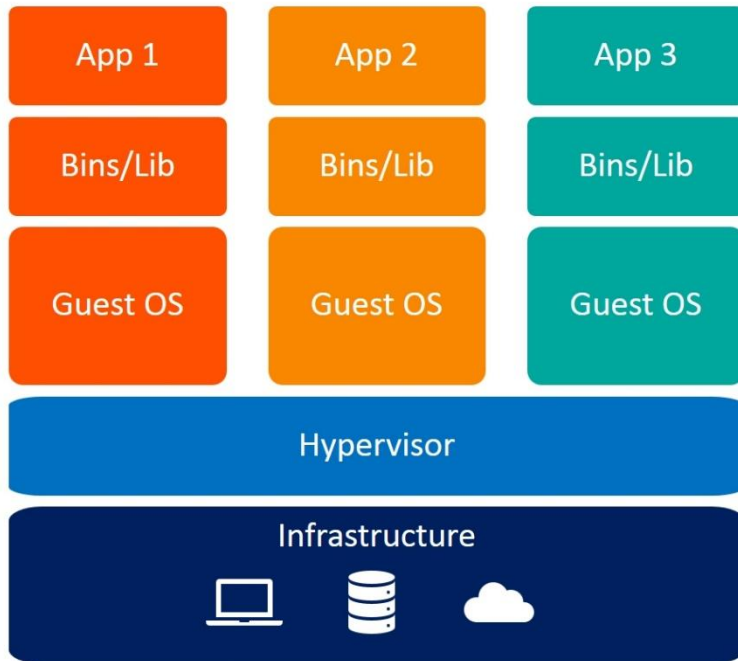


Béchir BEJAOUÏ
Formateur et consultant indépendant

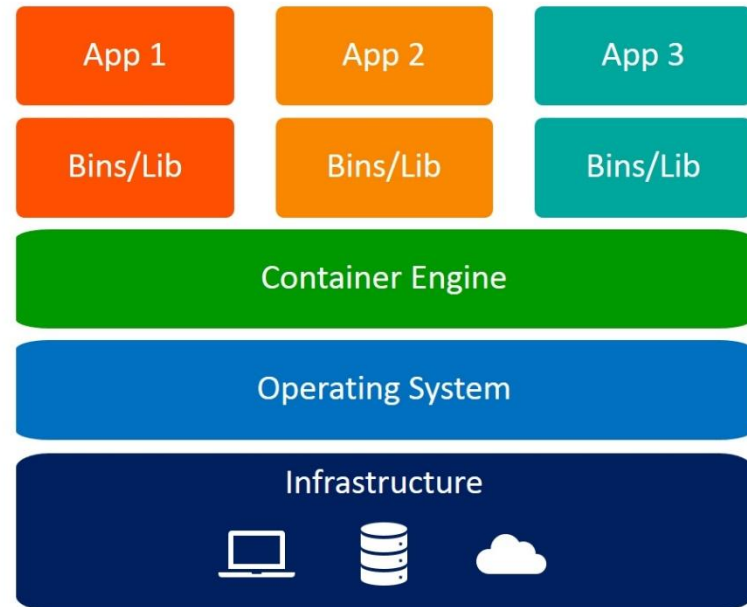
Le plan

- Présentation du Docker
- Préparation de l'environnement
- Le docker file
- La gestion des images et les conteneurs

Présentation du Docker (Virtualisation Docker vs Virtualisation VM)



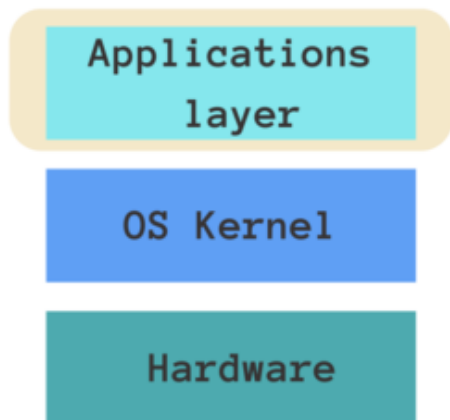
Virtual Machines



Containers

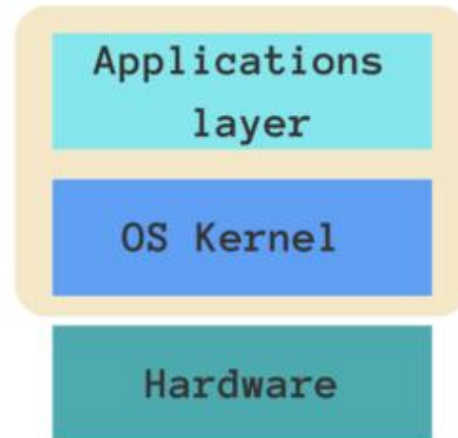
Présentation du Docker (Virtualisation Docker vs Virtualisation VM)

Container: Virtualization of the
Applications layer only



Sans docker

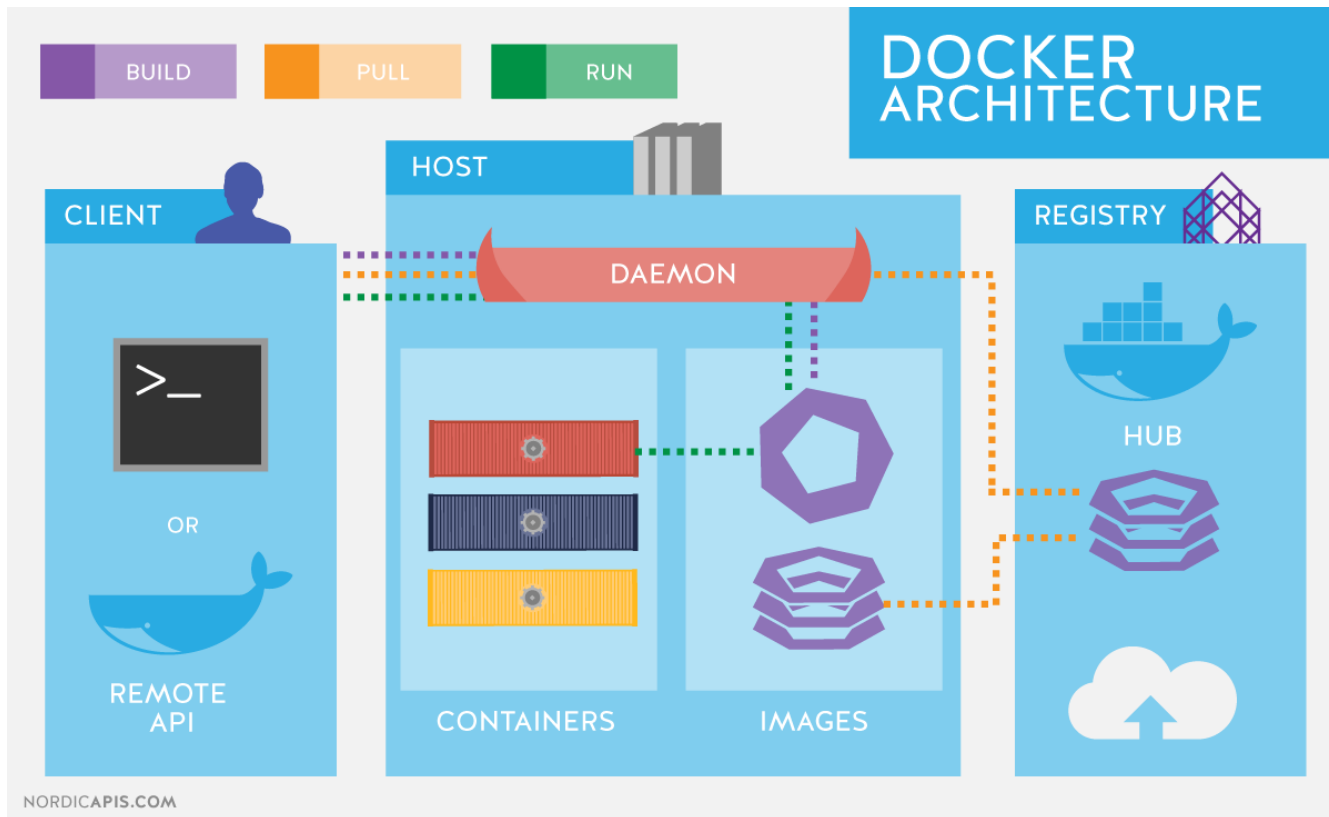
VM: Virtualization of the OS Kernel +
Applications layer



Avec docker

Présentation du Docker

- Docker est un projet open source permettant d'automatiser le déploiement d'applications en tant que conteneurs portables et autonomes exécutables sur le cloud ou localement



Installation Docker sous Linux

Installation Docker sous Linux Ubuntu

Installation:

```
sudo apt update  
sudo apt install apt-transport-https ca-certificates curl software-properties-common  
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add -  
sudo add-apt-repository "deb [arch=amd64] https://download.docker.com/linux/ubuntu focal stable"  
apt-cache policy docker-ce  
sudo apt install docker-ce
```

Post installation:

```
sudo systemctl status docker  
sudo usermod -aG docker ${USER}  
docker --version  
docker
```

Installation Docker sous Linux Centos

Installation:

```
sudo yum check-update  
curl -fsSL https://get.docker.com/ | sh  
sudo systemctl start docker
```

Post installation:

```
sudo systemctl status docker  
sudo systemctl enable docker  
sudo usermod -aG docker $(whoami)  
docker --version  
docker  
docker run hello-world
```

Installation Docker sous MAC

Préparation de l'environnement

- Dans le contexte du Mac OS, il faut visiter le lien <https://docs.docker.com/desktop/install/mac-install/>
- Il faut choisir le package convenable
 - Architecture Intel
 - Architecture MAC

Install Docker Desktop on Mac

Estimated reading time: 5 minutes

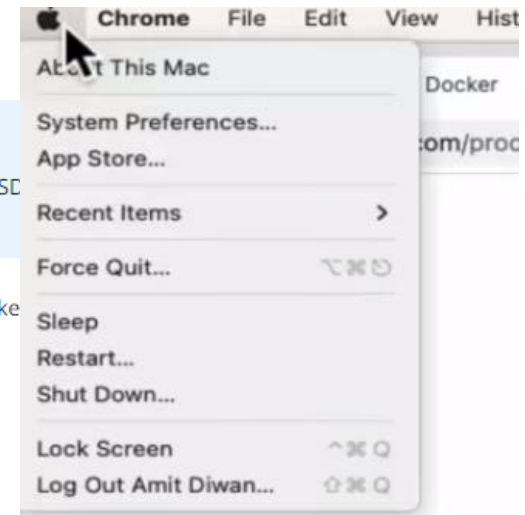
Docker Desktop terms

Commercial use of Docker Desktop in larger enterprises (more than 250 employees OR more than \$10 million USD) requires a paid subscription.

This page contains information about system requirements, download URLs, and instructions on how to install Docker Desktop.

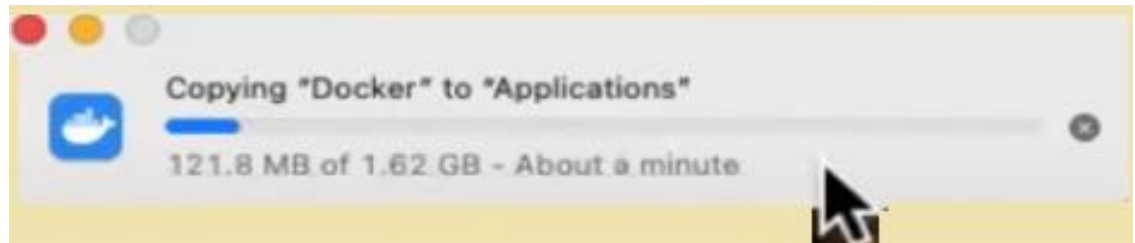
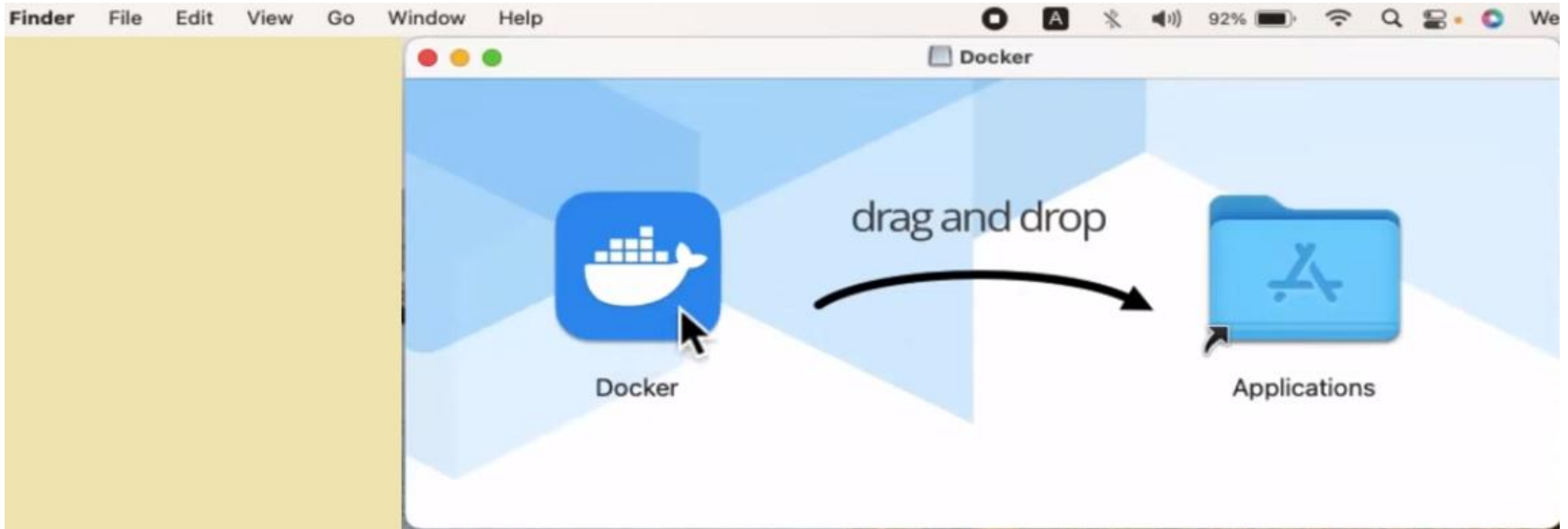
Mac with Intel chip

Mac with Apple chip



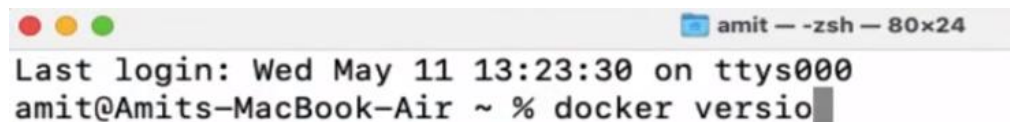
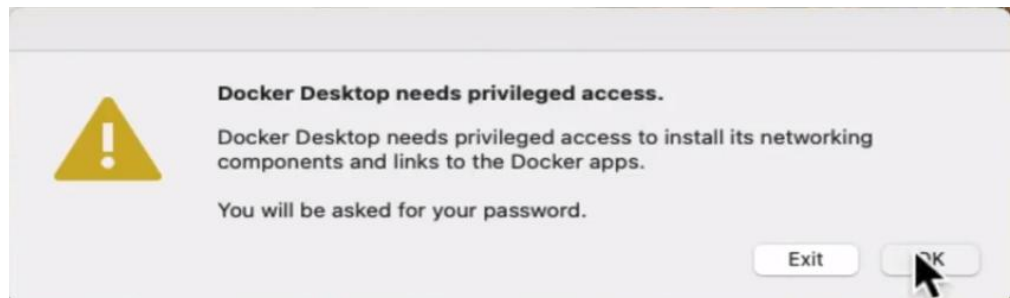
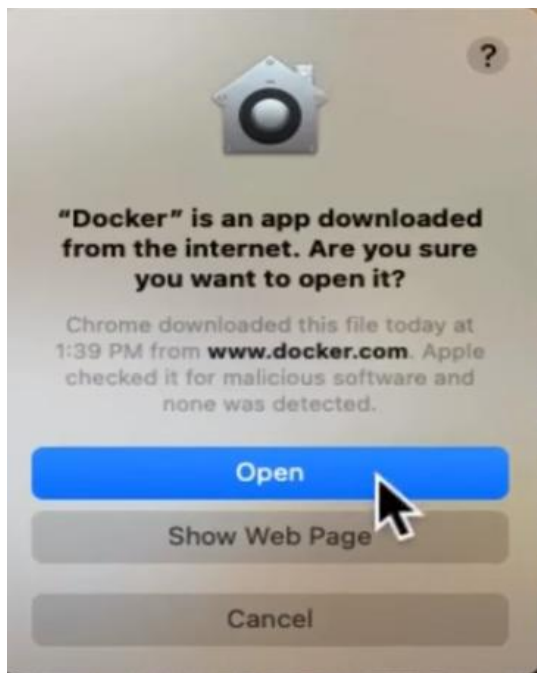
Préparation de l'environnement

- Télécharger le fichier dmg et glisser déposer le au niveau des applications



Préparation de l'environnement

- Toujours dans Mac OS lancer le **LaunchPad**, taper Docker et lancer le Docker
- Attribuer les droits d'accès

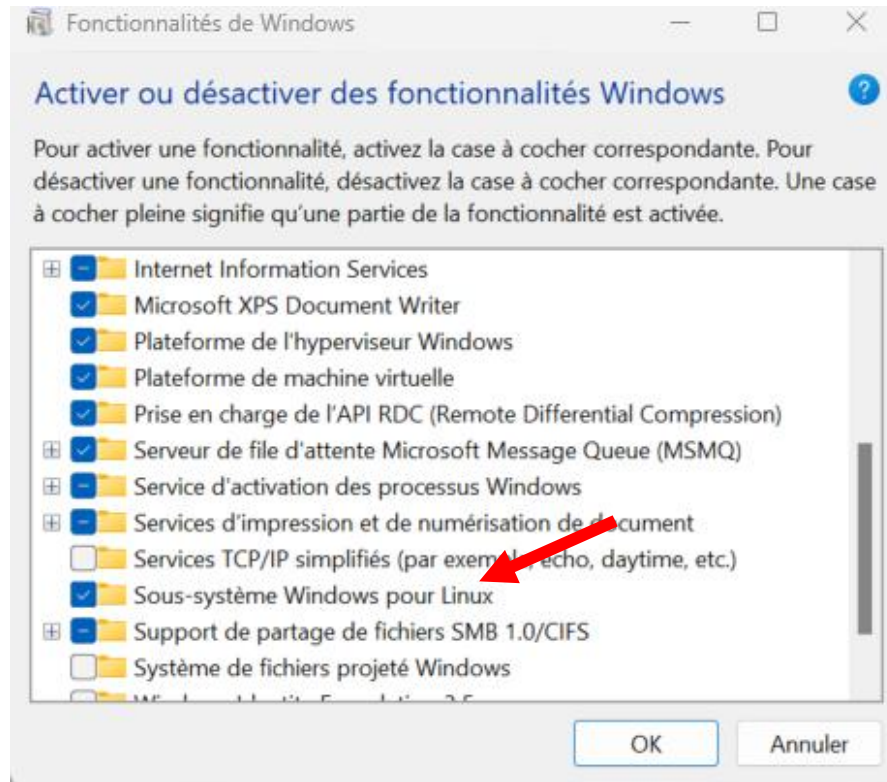


Installation Docker sous Windows

Avec Windows Sub System Linux

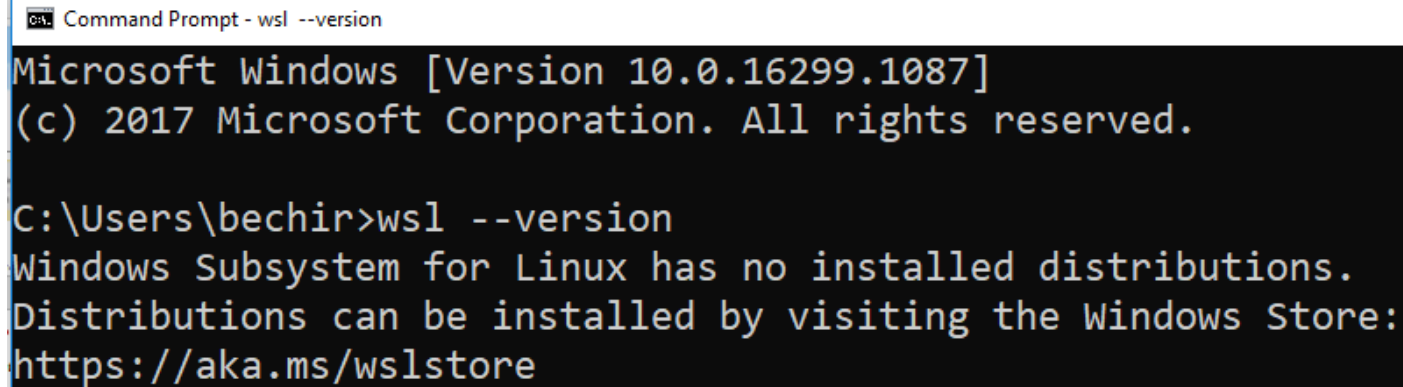
Préparation de l'environnement

- Dans un contexte Windows, il faut installer soit
 - Linux Subsystem **WSL**
- Pour installer Linux Subsystem, il faut ajouter la fonctionnalité à partir du panneau de configuration et redémarrer le système



Préparation de l'environnement

- Après redémarrage, il faut bien vérifier que la commande wsl est existante



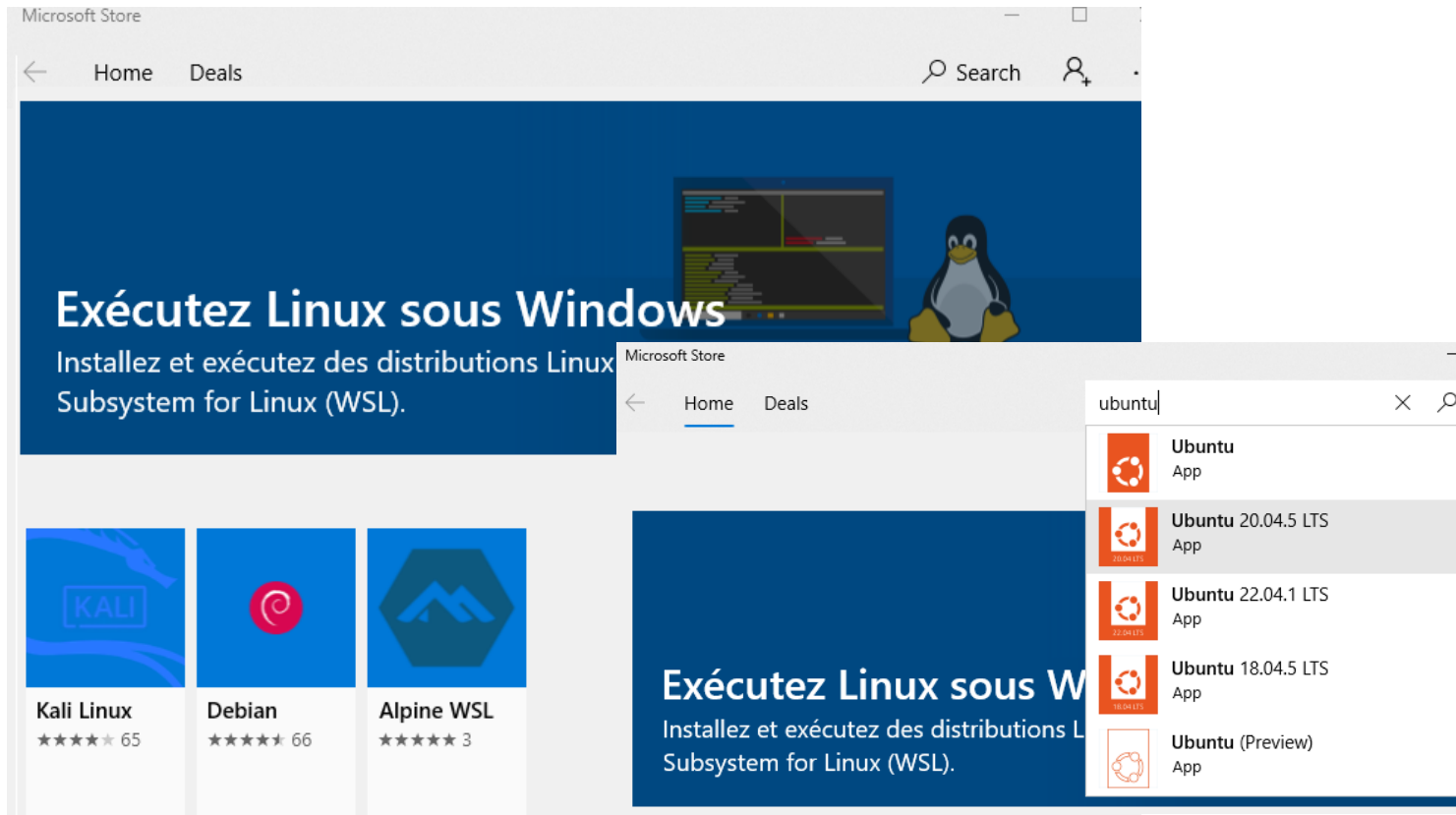
Command Prompt - wsl --version

```
Microsoft Windows [Version 10.0.16299.1087]
(c) 2017 Microsoft Corporation. All rights reserved.

C:\Users\bechir>wsl --version
Windows Subsystem for Linux has no installed distributions.
Distributions can be installed by visiting the Windows Store:
https://aka.ms/wslstore
```

Préparation de l'environnement

- Lancer en suite l'installation de la distribution Linux de choix, après avoir effectué l'identification au niveau du Windows Store



Préparation de l'environnement

- Lancer en suite l'installation de la distribution Linux de choix, après avoir effectué l'identification au niveau du Windows Store

 You own this app.

[Install](#) ...



Ubuntu 20.04.5 LTS

Canonical Group Limited • [Developer tools](#)

★★★★★ 20 [Share](#)

Install a complete Ubuntu terminal environment in minutes with Windows Subsystem for Linux (WSL). Develop cross-platform applications, improve your data science or web development

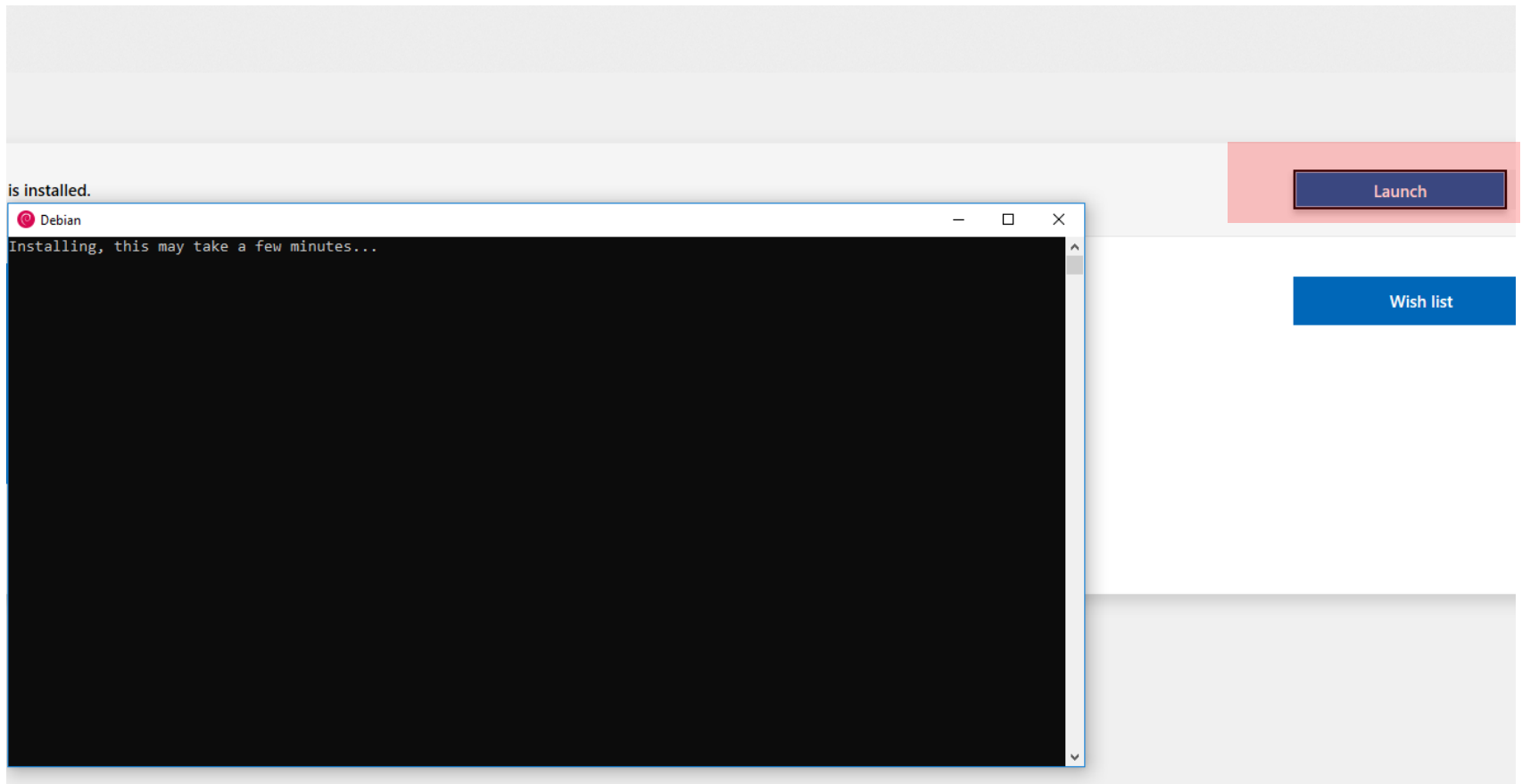
[More](#)

 PEGI 3

[Wish list](#)

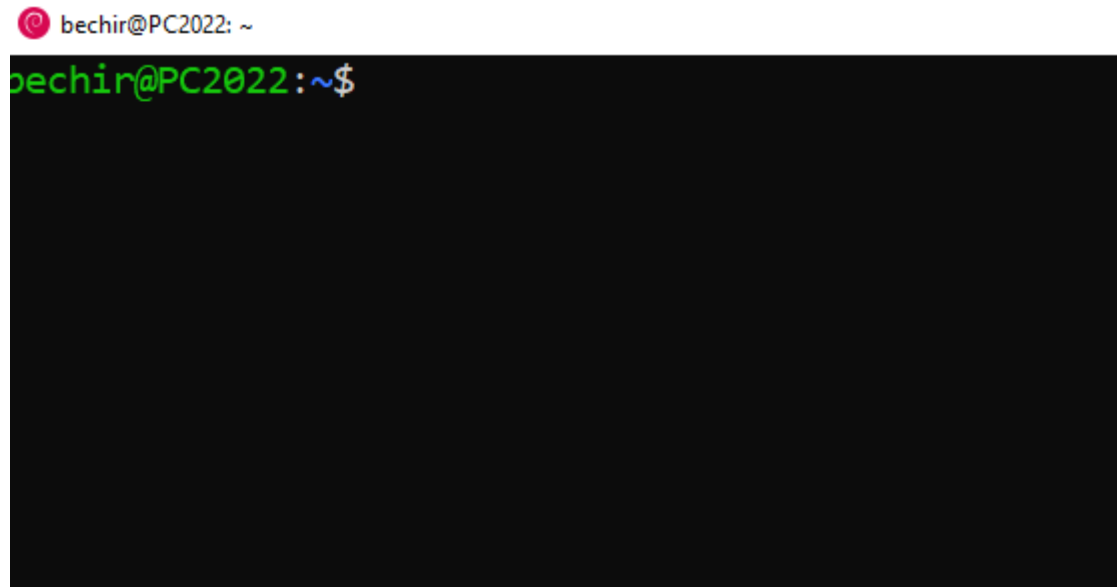
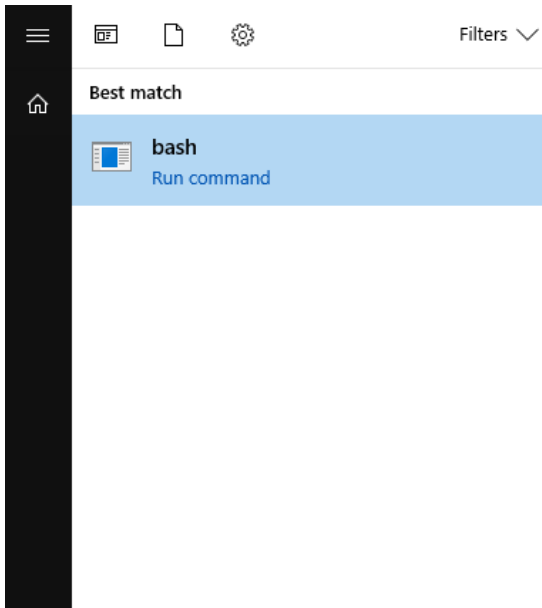
Préparation de l'environnement

- Lancer en suite l'installation de la distribution Linux de choix, après avoir effectué l'identification au niveau du Windows Store



Préparation de l'environnement

- Un fois la version est Linux est installée, il est possible de la lancer via l'utilitaire bash



Préparation de l'environnement

- L'étape suivante consiste à installer le Docker Desktop
<https://docs.docker.com/desktop/install/windows-install/>

Install Docker Desktop on Windows

Estimated reading time: 7 minutes

Docker Desktop terms

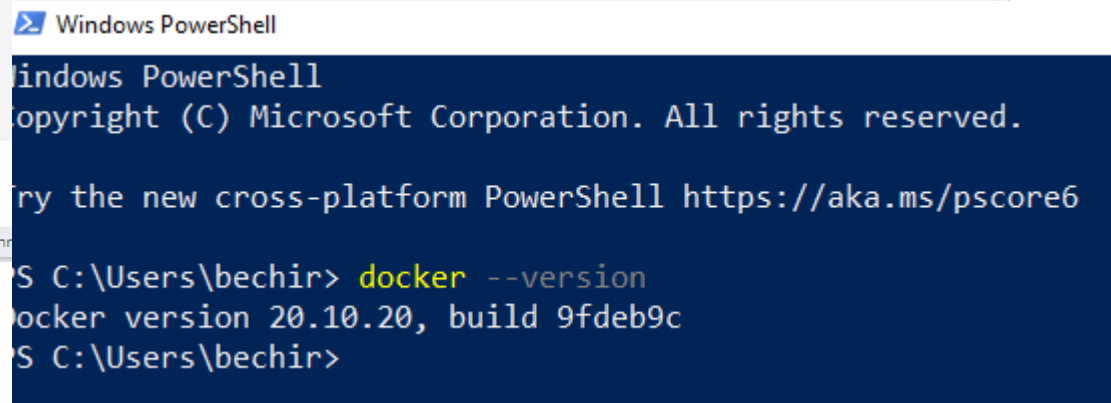
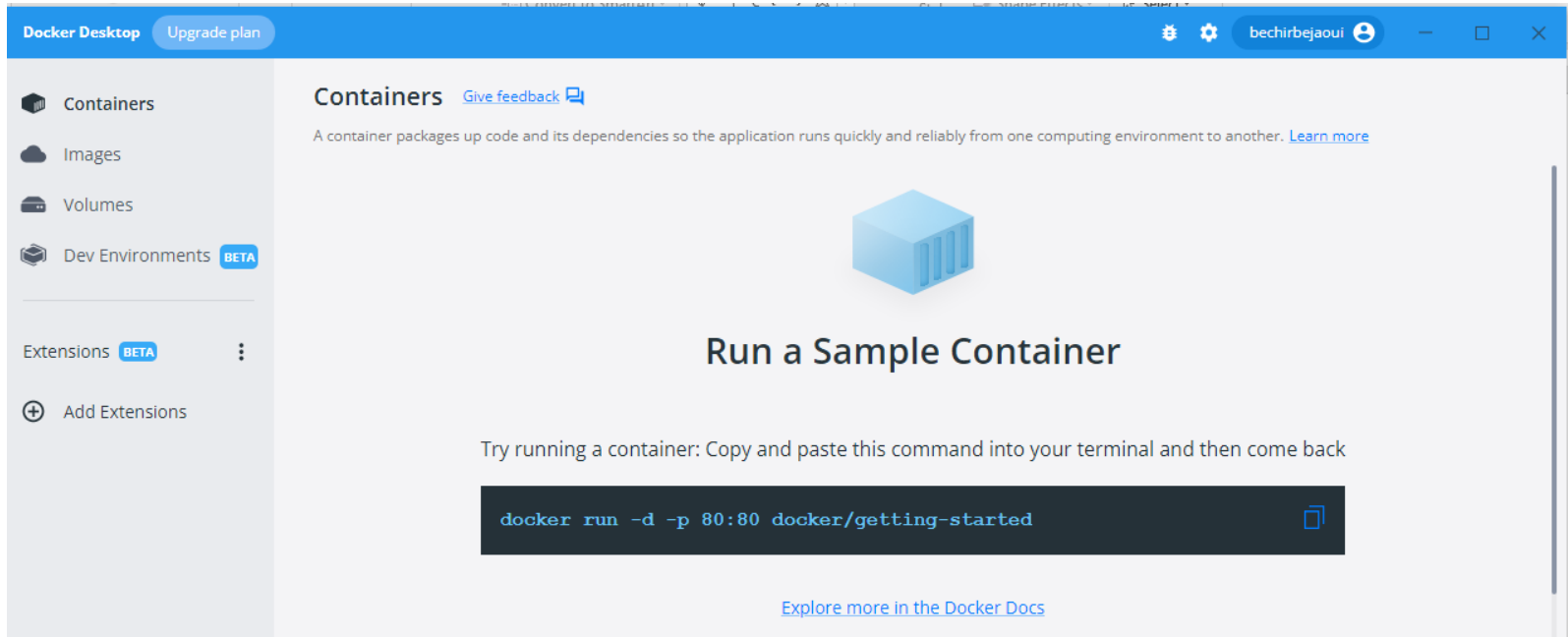
Commercial use of Docker Desktop in larger enterprises (more than 250 employees OR more than \$10 million USD in annual revenue) requires a paid subscription.

Welcome to Docker Desktop for Windows. This page contains information about Docker Desktop for Windows system requirements, download URL, instructions to install and update Docker Desktop for Windows.

Download Docker Desktop for Windows

[Docker Desktop for Windows](#)

Préparation de l'environnement



Architecture orientée service:

Ce support de cours est la propriété intellectuelle de Béchir Béjaoui, il n'est utilisé qu'avec l'accord du propriétaire © 2022-2023

Installation Docker sous Windows

Avec Chocolatey

Préparation de l'environnement

- Il faut installer Chocolatey

Installation:

Set-ExecutionPolicy AllSigned

ou

Set-ExecutionPolicy Bypass -Scope Process

```
Set-ExecutionPolicy Bypass -Scope Process -Force;  
[System.Net.ServicePointManager]::SecurityProtocol =  
[System.Net.ServicePointManager]::SecurityProtocol -bor 3072; iex ((New-  
Object  
System.Net.WebClient).DownloadString('https://chocolatey.org/install.ps1'))
```

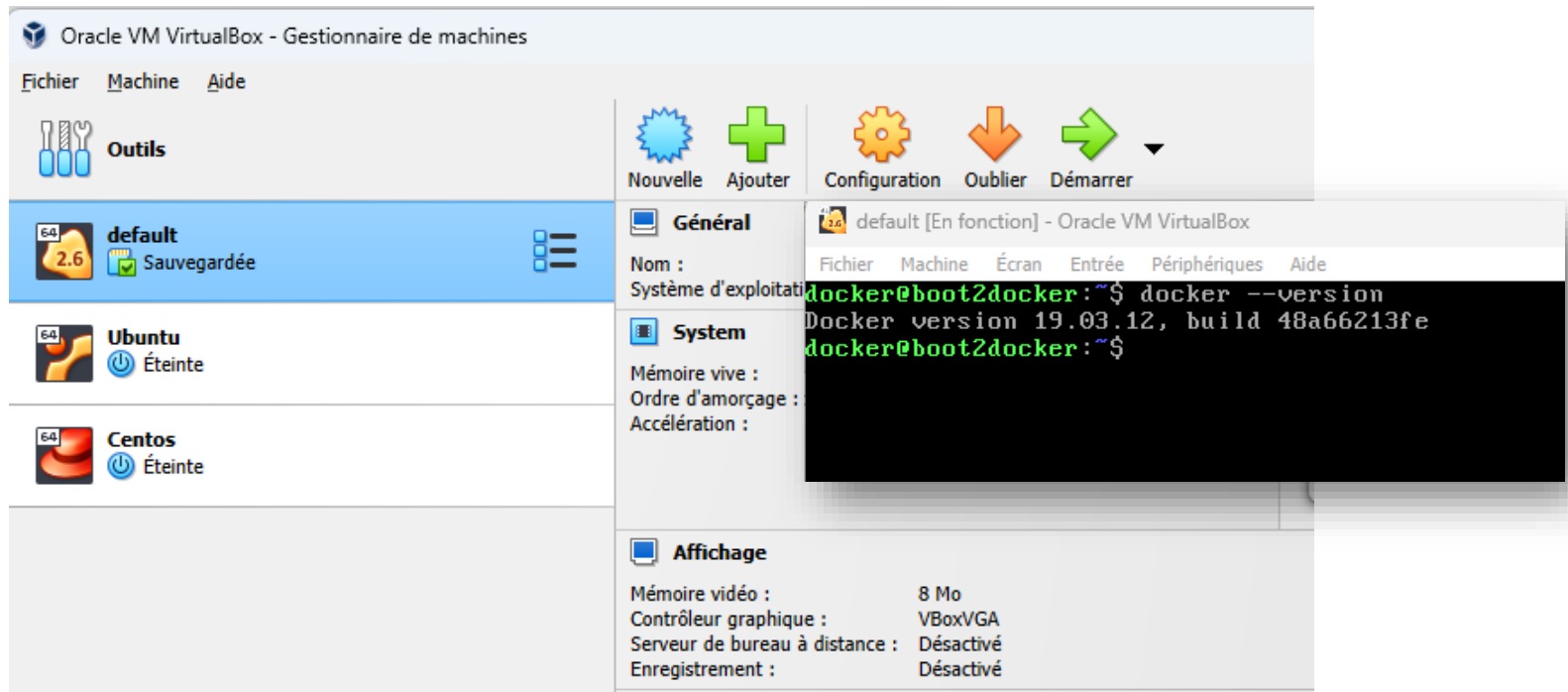
Post installation:

choco --version

Préparation de l'environnement

- Il faut installer Virtual Box et ajouter le chemin du Virtual Box à la variable d'environnement PATH

```
set PATH=%PATH%;"C:\Program Files\Oracle\VirtualBox"
```



Préparation de l'environnement

Il faut installer la machine docker

Activer le paramètre VTx dans le BIOS. Cela peut être vérifié et activé en appuyant sur certaines touches spécifiques comme F2 ou F10 ou autres au moment du démarrage du système

Créer la machine Docker:

docker-machine create --driver virtualbox default

Si erreur il faut exécuter la commande:

docker-machine create -d virtualbox --virtualbox-no-vtx-check default

Lancer la machine docker:

docker-machine env default | Invoke-Expression

Tester le docker:

docker run hello-world

Les commandes Docker

La gestion des images

#Chercher des images

sudo docker search sqlserver

sudo docker search httpd

sudo docker search phpmyadmin

#Lancer la commande de recherche

docker image --help

#Télécharger une image

docker pull httpd

#Télécharger une image pour une version particulière

docker pull httpd:2

#Lister les images

sudo docker images

#supprimer une image image

sudo docker image rm httpd:2

sudo docker rmi httpd:2

#Supprimer les images qui ne sont pas utilisées

docker image prune -a

#Construire une image

Sudo docker build -t . <- point

#Supprimer toutes les images

sudo docker rmi \$(docker images -a -q)

sudo docker rmi \$(docker images -ls -q)

#Inspecter des images

sudo docker inspect ubuntu

#Lister les images installées

docker image ls

docker images

#Sauvegarder une image dans un fichier tar

docker image save ubuntu -output|-o ubuntu.tar ubuntu

#Charger une image depuis un fichier tar

docker image load -input|-i ubuntu.tar

#Tagger une image avant de la déployer

docker image tag ubuntu bechirbejaoui/ubuntu:latest

#Pousser une image vers le Docker registry

docker image push bechirbejaoui/ubuntu:latest

#Pousser tout les tag image vers le Docker registry

docker image push -a bechirbejaoui/ubuntu

La gestion des conteneurs

#Créer un conteneur

sudo docker run --name m1 -tdi ubuntu

-t : Terminal;

-i : (interactif) ;

-d : (mode détaché : background mode) ;

#Lister les conteneurs en marche

sudo docker ps

#Lister les conteneurs

sudo docker ps -a

#Exécuter un conteneur

docker exec -ti m1 bash

#Lancer un conteneur

docker start m1

#Stopper un conteneur

docker stop m1

#Inspecter un conteneur

docker inspect m1

#Afficher les 20 dernières lignes des journaux d'un conteneur

docker logs - -tail=20 m1

#Exporter le système de fichiers vers l'extérieur

docker container export m1 --o filesystem.tar

#Supprimer un conteneur

docker rm m1

docker kill m1

#Supprimer tout les conteneurs non actifs

docker container prune

#Supprimer tout les conteneurs

docker stop \$(docker ps -aq)

docker rm \$(docker ps -aq)

La gestion des réseaux

#installer des utilitaires de test dans les conteneurs

apt-get update -y

apt-get install -y iputils-ping

br

#Lister les réseaux

docker network ls

#Inspecter un réseau

docker network inspect bridge

#créer un réseau

docker network create --driver bridge vlan1

#créer un réseau avec une @IP réseau et une @IP gateway

docker network create vlan2

--subnet=192.168.10.0/24

--gateway=192.168.10.1

#créer un conteneur avec un réseau existant

docker run -d --name --network -p -v image name

#connecter un conteneur à un réseau

docker network connect vlan1 srv1

#Déconnecter un conteneur d'un réseau

docker network disconnect vlan1 srv1

#Supprimer un réseau

docker network rm vlan1

#Supprimer les réseaux non utilisés

docker network prune -f

La gestion des volumes

#Lister les volumes

docker volume ls

#Créer un volume

docker volume create mysqlvolume

#Inspecter un réseau

docker volume inspect mysqlvolume

#Supprimer les volumes non utilisés

docker volume prune

#supprimer un volume

docker rm volume mysqlvolume

#Créer une instance docker sans volume

**docker run --name=mysqlc3 -p 127.0.0.1:3307:3306 -e
MYSQL_ROOT_PASSWORD=test123++ -tdi mysql**

#Créer une instance docker avec volume

**docker run --name=mysqlc3 -p 127.0.0.1:3307:3306
-v data:/var/lib/mysql
-e MYSQL_ROOT_PASSWORD=test123++ -tdi mysql**

Le Docker File

- Docker file est un élément indispensable pour la création des images docker qui donneront ensuite la vie aux applications micro services containerisées
- Le **Dockerfile** (toujours avec une majuscule) est un fichier qui contient toutes les instructions pour créer une image, comme des métadonnées (Mainteneur, label, etc.), ou même les commandes à exécuter pour installer un logiciel
- Voici la liste des instructions d'un Dockerfile les plus communément utilisées

FROM : Définit l'image de base qui sera utilisée par les instructions suivantes.

Pour multi FROM utiliser l'option - - **target**

Exemple: `docker build -t image1 --target os1 .`

LABEL : Ajoute des métadonnées à l'image avec un système de clés-valeurs, permet par exemple d'indiquer à l'utilisateur l'auteur du Dockerfile.

ARG : Variables temporaires qu'on peut utiliser dans un Dockerfile.

Le Docker File

ENV : Variables d'environnements utilisées dans le conteneur comme les mots de passes, les chaines de connexions.

RUN : Exécute des commandes Linux ou Windows lors de la création de l'image. Chaque instruction RUN va créer une couche en cache qui sera réutilisée dans le cas de modification ultérieure du Dockerfile.

COPY : Permet de copier des fichiers depuis la machine hôte vers le conteneur Docker.

ADD : Même chose que COPY mais prend en charge des liens ou des archives (si le format est reconnu, alors il sera décompressé à la volée).

Le Docker File

ENTRYPOINT : comme son nom l'indique, c'est la commande qui sera toujours exécutée au démarrage du conteneur. Elle prend la forme de tableau JSON (ex : CMD ["cmd1","cmd1"]) ou du texte.

CMD : Spécifie les arguments qui seront envoyés à l'ENTRYPOINT, (on peut aussi l'utiliser pour lancer des commandes par défaut lors du démarrage d'un conteneur). Si elle est utilisée pour fournir des arguments par défaut pour l'instruction ENTRYPOINT, alors les instructions CMD et ENTRYPOINT doivent être spécifiées au format de tableau JSON.

Note: **ENTRYPOINT** et **CMD** sont similaires à première vue mais elles ont deux comportements différents lors de l'exécution

Les commandes en provenance de **CMD** peuvent être écrasées par des nouvelles commandes **CMD** ou des commandes lancées lors de l'exécution du conteneur

Les commandes en provenance de **ENTRYPOINT** sont fixes et interchangeables

La combinaison de **ENTRYPOINT** et **CMD** est possible en se moment CMD jouera le rôle d'ajout d'options supplémentaires à **ENTRYPOINT**

Le Docker File

Exemple:

Soit les deux Dockerfile suivants:

```
FROM ubuntu
```

```
CMD ["echo", "Hello form cmd"]
```

```
FROM ubuntu
```

```
ENTRYPOINT ["echo", "Hello form entry point"]
```

Si on lance deux images issues des deux **Dockerfile** ce là va aboutir au même resultat

Docker run -ti <nom de l'image>

Si on teste la CMD

```
FROM ubuntu
```

```
CMD ["echo", "Hello form cmd"]
```

```
CMD ["echo", "Hello form cmd 2 "]
```

Le output sera dans ce cas **Hello form cmd 2**

Le Docker File

Exemple:

Si on teste avec

Docker run -it <image docker avec cmd> echo "Un nouveau message"

Ce là va afficher **Un nouveau message**

Par contre si on exécute l'image avec entry point

Docker run -it <image docker avec entry point> echo "Un nouveau message"

Ce là va afficher **Hello World echo Un nouveau message**

Il faut remarquer que echo est affichée en termes de message dans le message de sortie

Le Docker File

Une troisième alternative peut remplacer **CMD** et **ENTRYPOINT**, c'est **COPY** qui permet injecter un script pour qu'il soit exécuté au moment du lancement du conteneur

Exemple:

```
FROM mysql
```

```
# Set the environment variables
```

```
ENV MYSQL_ROOT_PASSWORD=test123++
```

```
ENV MYSQL_DATABASE=businessdb
```

```
ENV MYSQL_USER=test
```

```
ENV MYSQL_PASSWORD=test123++
```

```
COPY script.sql /docker-entrypoint-initdb.d/
```

Le Docker File

WORKDIR : Définit le répertoire du travail qui sera utilisé en mode sh ou bash

EXPOSE : Expose un port.

VOLUMES : Crée un ou des points de montage qui permettront de persister les données

USER : Désigne quel est l'utilisateur qui lancera les prochaines instructions RUN, CMD ou ENTRYPOINT

Le Docker Commit

C'est le processus inverse, c'est la création d'une image personnalisée à partir d'un conteneur modifié

Il ne prend pas en charge les volumes

Il arrête l'exécution du conteneur lors de la génération de l'image

La commande **nginx cp** est utilisée pour modifier le contenu de la page par défaut de nginx

docker cp index.html <nom conteneur>:/usr/share/nginx/html/

```
ubuntu@m1:~$ sudo vi index.html
ubuntu@m1:~$ docker cp index.html c1:/usr/share/nginx/html/index.html
Successfully copied 2.05kB to c1:/usr/share/nginx/html/index.html
```

Si le conteneur est redémarré il conservera les mêmes caractéristiques

Pour persister le changement sous forme d'image docker il faut utiliser la commande

docker commit <nom du conteneur> -m "Message "

Le Docker Commit

Une fois l'image est créée elle sera sans nom

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
<none>	<none>	119d55babf11	19 seconds ago	187MB
webimg	latest	683c1010f890	31 hours ago	187MB
gcr.io/k8s-minikube/kicbase	v0.0.42	dbc648475405	6 weeks ago	1.2GB

Il faut donc la tagger

docker tag <identifiant d'image> <nom d'image non existant déjà>

Il est également possible de créer directement une image à partir d'un conteneur de cette manière

docker commit <nom du conteneur> <nom de l'image>

Pour ajouter un commentaire pour expliquer les modifications

docker commit -m "message" <nom du conteneur> <nom de l'image>

Le Docker Commit

Pour modifier un paramètre

docker commit --change "Paramètre clé/valeur" <nom du conteneur> <nom de l'image>

Exemple

Commencer par créer deux fichiers:

index.html de test:

```
<html><body>  
    <h1>Page de test</h1>  
</body></html>
```

nginx.conf pour changer le port par défaut

Voir la partie des notes

Lancer un conteneur vierge

docker run -d --name <nom du conteneur> -p 80:80 nginx

Changer le contenu web

docker cp index.html <nom du conteneur> :/usr/share/nginx/html/index.html

Changer la configuration

docker cp nginx.conf <nom du conteneur> :/etc/nginx/nginx.conf

Le Docker Commit

Confirmer le changement de la configuration

docker commit --change "EXPOSE 8000" <nom du conteneur>

Tagger la nouvelle image

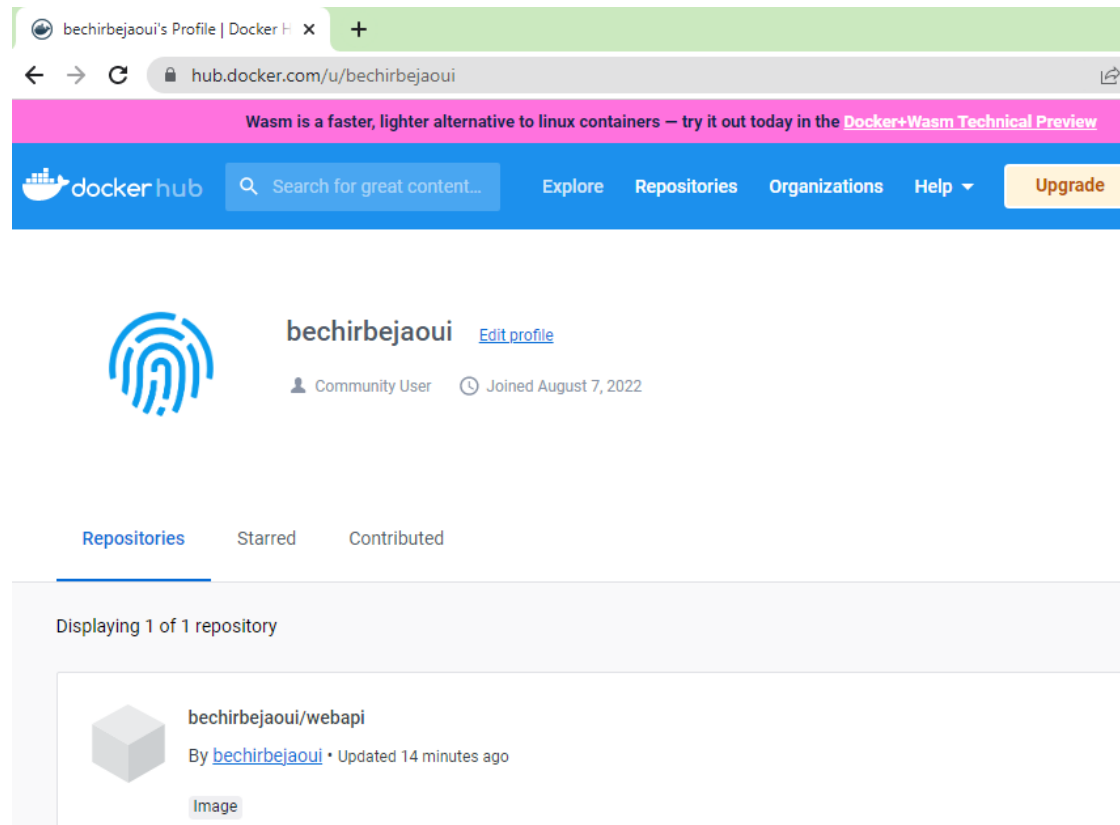
docker tag <identifiant d'image> <nom d'image non existant déjà>

Créer un nouveau conteneur

docker run -d --name <nom du nouveau conteneur> -p 8000:8000 <nom de l'image>

La gestion des images

Le **Registry**, c'est l'implémentation open source permettant de stocker et de distribuer des images de conteneurs. Le docker hub est le principal entrepôt des images docker



Architecture orientée service:

Ce support de cours est la propriété intellectuelle de Béchir Béjaoui, il n'est utilisé qu'avec l'accord du propriétaire © 2022-2023

La gestion des images (Docker hub)

Pour stocker les images dans le hub il faut avoir un compte

L'authentification se fait vers docker hub à l'aide de la commande
docker login -u <nom utilisateur> -p <mot de passe>

L'authentification se fait vers les autres registry à l'aide de la commande
docker login -u <nom utilisateur> -p <mot de passe> <nom du registry>

Il faut ensuite tager l'image

docker --tag <nom local d'image> <nom du compte>/ <nom local d'image>

Enfin

docker push

La gestion des images (Gitlab)

Il faut tout d'abord créer un **Token** Personnel avec les droits de lecture et d'écriture de et vers le **Container Registry** de **Gitlab**

The screenshot shows the GitLab user interface. On the left, the user profile menu is open, with 'Preferences' highlighted. On the right, the 'User settings' menu is open, with 'Access Tokens' highlighted. Below this, the 'Active personal access tokens' section is shown, containing a table with one token. The 'Add new token' button is also highlighted.

Token name	Scopes	Created	Last Used	Expires	Action
ACCESS_TOKEN	api, read_api, read_user, create_runner, k8s_proxy, read_repository, write_repository, read_registry, write_registry	Dec 16, 2023	4 days ago	in 3 weeks	

La gestion des images (Gitlab)

Il faut tout d'abord créer un **Token** Personnel avec les droits de lecture et d'écriture dans le registry et la lecture des API vers le **Container Registry** de **Gitlab**

Token name

ACCESS_TOKEN

For example, the application using the token or the purpose of the token.

Expiration date

2024-01-19

Select scopes

Scopes set the permission levels granted to the token.

- ☐ api
- ☒ read_api
- ☐ read_registry
- ☐ write_registry
- ☐ read_user
- ☐ create_runner
- ☐ k8s_proxy
- ☐ read_repository
- ☐ write_repository
- ☐ ai_features

Create personal access token

Cancel

read_api

Grants read access to the API, including all groups and projects, the container registry, the dependency proxy, and the package registry.

read_registry

Grants read-only access to container registry images on private projects.

write_registry

Grants write access to container registry images on private projects.

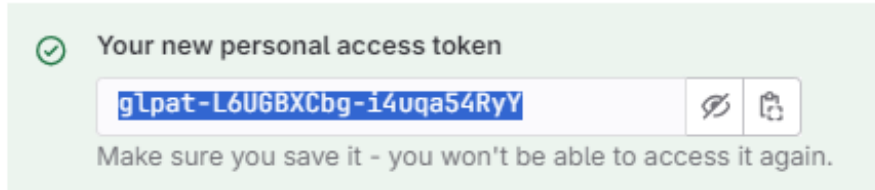
✓ Your new personal access token

glpat-L6UGBXCbg-14uqa54RyY

Make sure you save it - you won't be able to access it again.

La gestion des images (Gitlab)

Il faut garder le Token quelques parts car il va être montré une seule fois



En suite, il faut passer par trois phases pour publier les images vers gitlab

Il faut tagger l'image avec le nom complet de **plateforme/projet/groupe/<nom d'image>**

Exemple:

docker tag monimage registry.gitlab.com/bechir-test-group/dockerproject/monimage

Ensuite, il faut s'authentifier au niveau de la plateforme

Exemple :

docker login -u bechir888 -p glpat-L6UGBXCbg-i4uqa54RyY registry.gitlab.com

La gestion des images (Gitlab)

La troisième étape et de pousser vers le Gitlab

Exemple de poussée d'image vers gitlab

docker push registry.gitlab.com/bechir-test-group/dockerproject/monimage

```
ubuntu@m1:~$ docker push registry.gitlab.com/bechir-test-group/dockerproject/monimage
Using default tag: latest
The push refers to repository [registry.gitlab.com/bechir-test-group/dockerproject/monimage]
a1360aae5271: Pushing [=====>] 11.96MB/77.85MB
```

Container Registry

📦 1 Image repository ⌚ Cleanup is not scheduled.

⋮ dockerproject/monimage 📦

1 tag

La gestion des images (Gitlab)

Voici un exemple comment générer et publier une image depuis le code source en Gitlab en utilisant des variables

```
build-job:    # This job runs in the build stage, which runs first.
  image: docker
  services:
    - name: dind

  before_script:
    - echo $CI_REGISTRY_PASSWORD | docker login -u $CI_REGISTRY_USER $CI_REGISTRY --password-stdin

  script:
    - docker build -t $CI_REGISTRY_IMAGE/$CI_IMAGE:latest db/.
    - echo "image has been built successfully"
    - docker push $CI_REGISTRY_IMAGE/$CI_IMAGE:latest
    - echo "image has been pushed successfully"
```

Note: CI_IMAGE est une variable Gitlab personnalisée le reste des variables sont Des variables système Gitlab

Le Docker compose

Docker Compose est un outil permettant de définir le comportement de vos conteneurs et d'exécuter des applications Docker à conteneurs multiples.

La config se fait à partir d'un fichier YAML, et ensuite, avec une seule commande, vous créez et démarrez tous vos conteneurs de votre configuration.

```
version: '3.3'
services:
  db:
    image: mysql:5.7
    restart: always
    environment:
      MYSQL_PASSWORD: 'test123++'
      MYSQL_ROOT_PASSWORD: 'test123++'
    ports:
      - '127.0.0.1:3308:3306'
    expose:
      - '3306'
    volumes:
      - my-db:/var/lib/mysql
```

```
volumes:
  my-db:
```

Créer le conteneur:

docker compose up -d

Supprimer le conteneur:

docker compose down

Le Docker compose (Installation sous Ubuntu)

```
curl -SL https://github.com/docker/compose/releases/download/v2.3.3/docker-compose-linux-x86_64 -o ~/.docker/cli-plugins/docker-compose
```

```
chmod +x ~/.docker/cli-plugins/docker-compose
```

```
docker compose version
```

Le Docker compose

[GitHub - docker/compose: Define and run multi-container applications with Docker](#)

Exemple de docker compose documentation officielle

<https://docs.docker.com/compose/gettingstarted/>

Exemple

<https://devhints.io/docker-compose>